Text Classification:

Text classification is the task of assigning a predefined category or label to a piece of text. The core idea is identical to what I did with the Iris flowers, we have inputs (features) and we predicting an output (label) except now the input is unstructured text instead of clean numeric measurements which means it needs to be converted into a numeric form before any model can actually use it. That conversion step is basically the whole game in text classification; once our text is turned into numbers, the training/prediction/evaluation steps look almost identical to what I already did.

A few examples of text classification are:

- **Sentiment classification**: deciding whether a review, tweet or comment is positive or negative (sometimes neutral too).
- **Legal topic classification**: automatically tagging legal documents or clauses by the area of law they belong to (contracts, IP, employment, etc.) which is useful for legal research tools.
- **News category classification**: sorting news articles into sections like sports, politics, technology or business.
- **Complaint classification**: routing customer complaints to the right department based on what they're actually about (billing, delivery, product defect, etc.).
- **Spam detection**: deciding whether an email or message is spam or not.

NLP Preprocessing:

Text data is messy in ways numeric data usually isn't i.e. inconsistent casing, punctuation, filler words, different forms of the same word ("running" vs "run"). Before any model can meaningfully learn from text, it usually needs to be cleaned and standardized.

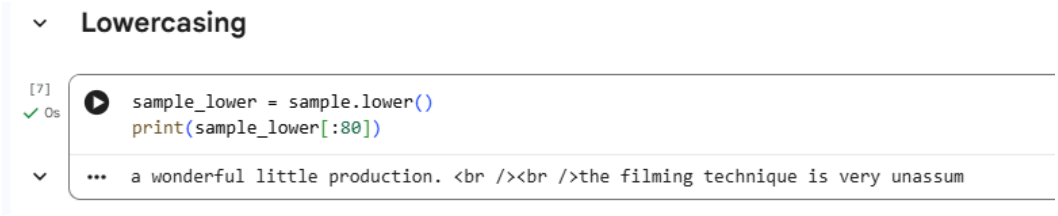
Here's the basic pipeline, applied to a sample IMDB review dataset taken from Kaggle:

Code and Screenshot:

Lowercasing:

The first and simplest step: convert everything to lowercase, so "Movie", "movie", and "MOVIE" are all treated as the same word instead of three different ones.

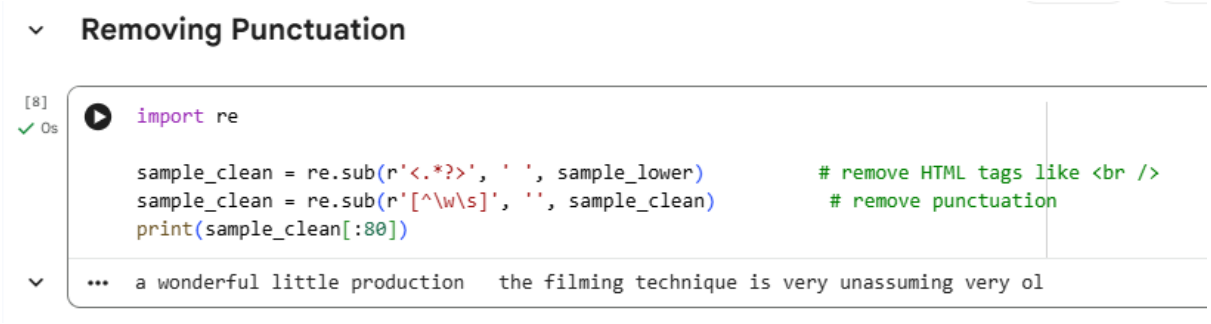
Code and Screenshot:



Removing Punctuation:

Punctuation marks (periods, commas, quotes, the leftover HTML tags in this dataset, etc.) usually add noise rather than meaning for basic classification tasks, so they get stripped out.

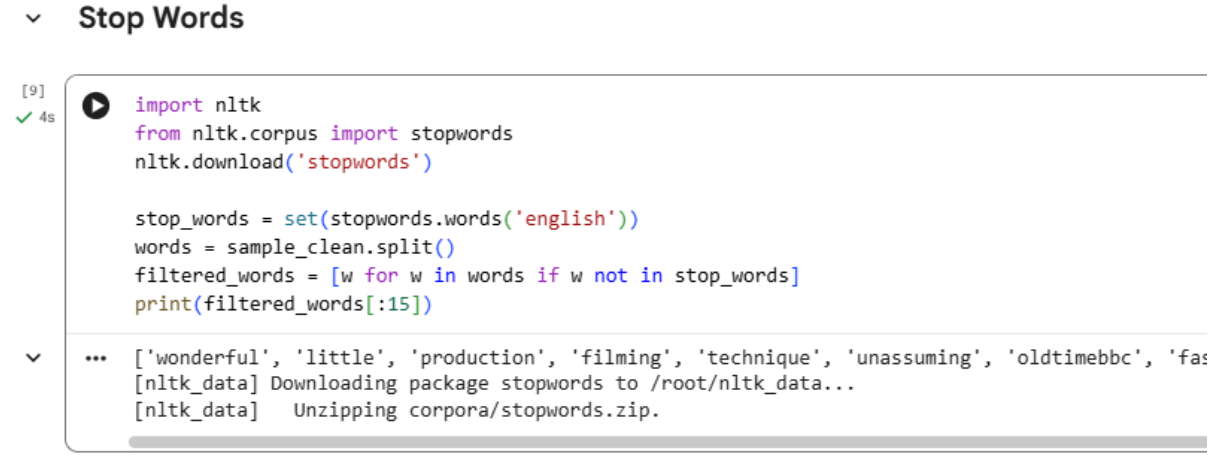
Code and Screenshot:



Stop Words:

Stop words are extremely common words i.e. "the," "is," "a," "and," "to" that appear constantly but usually carry very little useful signal for tasks like classification. Removing them shrinks the vocabulary and lets the model focus on words that actually distinguish one class from another.

Code and Screenshot:



Tokenization

Tokenization is the process of breaking text into individual units usually words, sometimes sub words or sentences that a model can work with. In the steps above I have actually already been tokenizing informally by splitting on whitespace but proper tokenizers handle edge cases better (contractions, punctuation attached to words, etc.).

Code and Screenshot:

Tokenization

[11] ✓ 0s

```
import nltk
nltk.download('punkt_tab')
from nltk.tokenize import word_tokenize
nltk.download('punkt')

tokens = word_tokenize(sample_clean)
print(tokens[:10])
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
['a', 'wonderful', 'little', 'production', 'the', 'filming', 'technique', 'is', 'very', 'unassuming']
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

Stemming / Lemmatization:

Words like "running," "runs," and "ran" are all different forms of the same root idea. Without some normalization, a model would treat them as three completely unrelated words, splitting the signal that should really be counted together.

- **Stemming** chops words down to a rough root by cutting off suffixes using fairly crude rules. It is fast but sometimes produces non-words (e.g., "studies" → "studi").
- **Lemmatization** is the more careful version, it uses vocabulary and grammar rules to reduce a word to its actual dictionary form (e.g., "studies" → "study", "better" → "good"). It is slower but more linguistically correct.

Code and Screenshot:

Stemming / Lemmatization

[12] ✓ 4s

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
nltk.download('wordnet')

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

words_to_test = ["running", "studies", "better", "wonderful"]

for w in words_to_test:
    print(w, "->", "stem:", stemmer.stem(w), "| lemma:", lemmatizer.lemmatize(w))
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
running -> stem: run | lemma: running
studies -> stem: studi | lemma: study
better -> stem: better | lemma: better
wonderful -> stem: wonder | lemma: wonderful
```

References:

https://www.geeksforgeeks.org/nlp/text-preprocessing-for-nlp-tasks/

https://www.geeksforgeeks.org/nlp/what-is-text-classification/

https://www.geeksforgeeks.org/nlp/natural-language-processing-nlp-tutorial/